

Operation of Real Mode Interrupt

1. Recognition Mechanism

Concept

Before an interrupt is serviced, the microprocessor must recognize its presence during its standard instruction cycle.

Rules / Notes

The Checking Sequence

1. The microprocessor **completes executing the current instruction**.
2. It then determines whether an **interrupt condition is active** (Hardware pins or internal flags).

2. Internal Execution Sequence (Real Mode)

If one or more interrupt conditions are present, the CPU follows this exact 5-step hardware sequence to preserve the program state:

Rules / Notes

The 5-Step Stack Save Process

1. **Flags Register:** Contents are pushed onto the stack.
2. **Control Flags:** Both **IF** (Interrupt) and **TF** (Trap) flags are cleared.
3. **Code Segment (CS):** Contents are pushed onto the stack.
4. **Instruction Pointer (IP):** Contents are pushed onto the stack.
5. **Vector Fetch:** Interrupt vector contents are fetched from the IVT and placed into **IP** and **CS**.

3. Return Address Logic

Concept

The return address pushed onto the stack (**CS:IP**) varies based on the type of interrupt that occurred.

Rules / Notes

Where does the Return Address point?

1. **Case 1: Points to where the interrupt occurred.**

This happens for interrupts that indicate an error or require re-execution.

- **Applies to:** Type 0, 5, 6, 7, 8, 10, 11, 12, and 13.

2. **Case 2: Points to the next instruction.**

Standard behavior for typical software/hardware requests.

- **Applies to:** All other interrupts.

4. Protected Mode Migration

As architecture evolved (80286+), the fixed-address IVT was replaced by a more flexible **Interrupt Descriptor Table (IDT)**.

Concept

IDTR (Interrupt Descriptor Table Register)

A **48-bit register** that defines the location and size of the IDT in memory.

- **IDT Base Address (32-bit):** Specifies the start of the IDT.
- **IDT Limit (16-bit):** Specifies the size of the table ($\text{Limit} = \text{Size} - 1$).

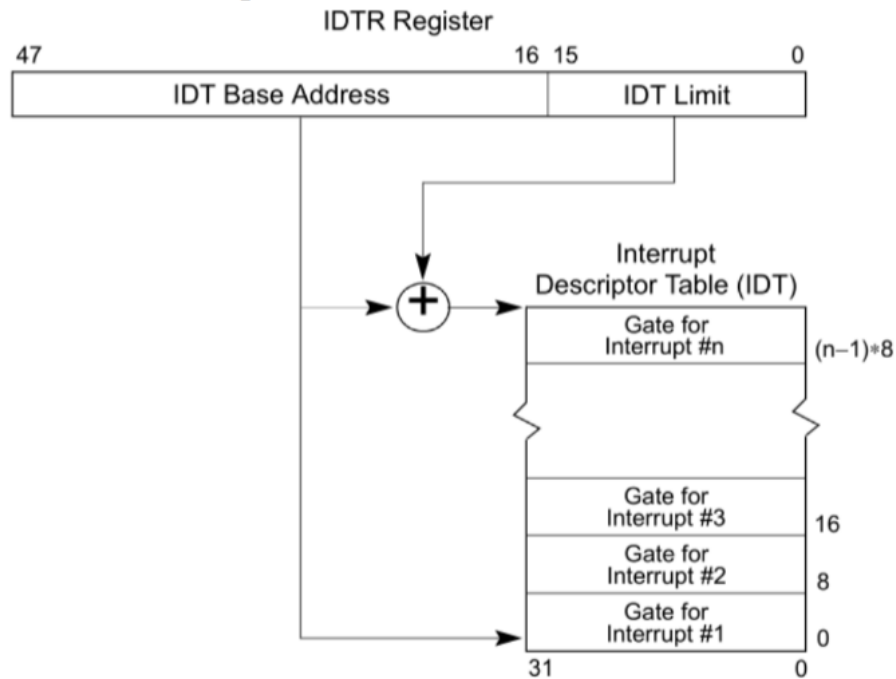
Rules / Notes

The Interrupt Descriptor Table (IDT)

- **Same Assignments:** Interrupts have exactly the same assignments as in Real Mode.
- **Descriptor Set:** A set of **256 interrupt descriptors** are used in place of interrupt vectors.
- **Storage:** Descriptors are stored in the Interrupt Descriptor Table (IDT).
- **Relocatable:** The IDT is located at any memory location in the system by the **IDTR**.
- **Descriptor Size:** Each descriptor contains **8 bytes**.
- **Table Size:** The IDT is $256 \times 8 = 2048$ bytes long.

4.1 IDT Addressing Mechanism (Figure Analysis)

Protected mode operation →



Concept

How the CPU locates a Descriptor:

1. **Base Address:** The CPU fetches the **32-bit IDT Base Address** from the IDTR.
2. **Scaling $((n - 1) \times 8)$:** Since each descriptor is **8 bytes**, the interrupt type number (#n) is adjusted to a 0-based offset by using the formula $(n - 1) \times 8$.
3. **Addition:** The result is added to the Base Address to find the exact starting location of the descriptor.
4. **Limit Check:** The CPU ensures that the calculated offset $((n - 1) \times 8)$ does not exceed the **16-bit IDT Limit**.

Rules / Notes

Comparison of Offset Calculations

- **Real Mode (IVT):** Pointer Address = $n \times 4$.
- **Protected Mode (IDT):** Descriptor Address = Base + $((n - 1) \times 8)$.

Exam Tips

Note on Indexing: Why $(n - 1) \times 8$?

In the lecture figure, the interrupts are labeled as **#1**, **#2**, ..., **#n** (1-based indexing).

- **Interrupt #1** corresponds to **Type 0**.
- **Interrupt #2** corresponds to **Type 1**.
- Therefore, for the ***n*-th** interrupt in the list, the offset is $(n - 1) \times 8$.

In technical programming (0-based), for a **Type *x*** interrupt, the offset is simply $x \times 8$.

Rules / Notes

Real Mode vs. Protected Mode Operation

- **Processor Support:**
 - **8086 and 8088:** Restricted to **Real Mode** only.
 - **80286 and above:** Can operate in either **Real or Protected Mode**.
- **Real Mode Operation:**
 - Addresses only the **first 1M byte** of memory space.
 - Allows **upward compatibility** for software written for 8086/8088.
- **Protected Mode Operation:**
 - Addresses within and **above the first 1M byte** (up to 4 GB).
 - Uses a **32-bit offset address** to access information.
 - Segments can be up to **4G bytes** in length.
 - Segment register contains a **selector** instead of a segment address.
- **Addressing Mechanism (Protected Mode):**
 - **Selector:** Selects a **descriptor** from the descriptor table.
 - **Descriptor:** Describes the memory segment's **location, length, and access rights**.

Example

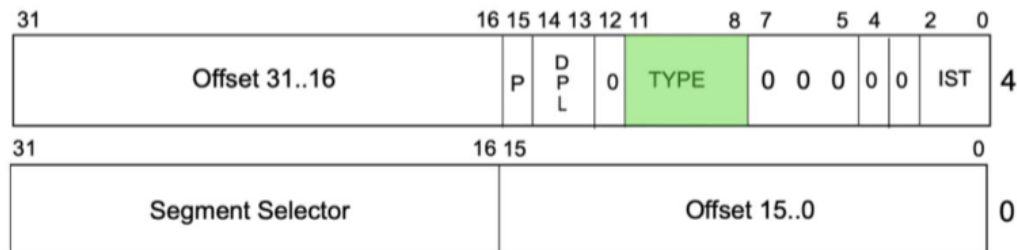
Transition Flow: Conversion to IDT

1. Copy Real Mode vector addresses (IP:CS).
2. Convert to **32-bit offsets** in the Protected Mode descriptor.
3. Load the IDTR with the new table address.

5. Operation of Protected Mode Interrupt

In Protected Mode, the microprocessor uses **Interrupt Descriptors** instead of vectors.

Structure of each entry in IDT →



Concept

Structure of an 8-Byte IDT Entry (Gate)

- **Upper 4 Bytes:**

- **Bits 31–16:** Offset 31..16
- **Bit 15:** P (Present bit)
- **Bits 14–13:** DPL (Privilege Level)
- **Bit 12:** 0 (Reserved)
- **Bits 11–8:** TYPE (Gate Type)
- **Bits 7–3:** Reserved (Must be 0)
- **Bits 2–0:** IST (Interrupt Stack Table)

- **Lower 4 Bytes:**

- **Bits 31–16:** Segment Selector
- **Bits 15–0:** Offset 15..0

Rules / Notes

Meaning of Each Field

1. **Offset (32-bit):** Split into 2 parts. Represents the address of the entry point of the **Interrupt Service Procedure (ISP)**.
2. **Selector:** Segment selector. Must point to a valid code segment.
3. **Gate Type (4-bit):** Defines the gate type this Interrupt Descriptor represents (**Task gate, interrupt gate, trap gate, or call gate**).
4. **DPL (Descriptor Privilege Level):** A 2-bit value that describes the privilege level of the interrupt.
5. **P (Present bit):** Must be set (1) for the descriptor to be valid.
6. **IST (Interrupt Stack Table):** Introduced in x86-64 (Long Mode). Used when an interrupt occurs during stack corruption or overflow.

Rules / Notes

Real Mode to Protected Mode Conversion

Real mode interrupt vectors can be converted into protected mode interrupts using these steps:

1. **Copy:** Extract the interrupt procedure addresses from the Real Mode IVT.
2. **Convert:** Translate these addresses into **32-bit offset addresses**.
3. **Store:** Place these new offsets, along with selectors and gate attributes, into the **Interrupt Descriptors**.

Concept

Global Descriptor Table (GDT) Role

The GDT identifies the **first 1M byte** of memory as the interrupt segment using a **Single Selector + Segment Descriptor**.

Rules / Notes

Operational Summary

- **Similarity:** Protected Mode interrupt functions like a Real Mode interrupt.
- **Access:** The microprocessor accesses the **IDT** instead of the IVT.
- **Return:** Both Real and Protected Mode interrupts return using **IRET** or **IRETD** instructions.

Example

Accessing the IDT

When an interrupt occurs (n):

1. The CPU calculates the descriptor address: $\text{Base} + ((n - 1) \times 8)$.
2. It fetches the 8-byte descriptor.
3. It switches to the segment indicated by the **Selector** and jumps to the address given by the **Offset**.

Exam Tips

Exam Tip: The Return Instruction

- **IRET**: Used in Real Mode (restores 6 bytes).
- **IRETD**: Used in Protected Mode (restores 12 bytes: EIP, CS, and EFLAGS).